

Docker Setup for Playwright + TypeScript Automation Framework

Overview

This Docker configuration containerises the Playwright test automation framework so tests run identically on **Windows**, **macOS**, and **CI/CD** environments — no manual browser or dependency installation needed.

The project uses the official Playwright container image:

```
mcr.microsoft.com/playwright:v1.62.0-noble
```

This base image includes the Playwright browser runtime, all required system dependencies, and the Chromium browser — everything is pre-installed and ready to go.

Architecture

Host (Windows / macOS / Linux)

```
docker compose run --rm playwright-tests <suite>
```



Container: playwright-opencart

docker-entrypoint.sh → maps suite name
to npm script



/app/

└─ tests/	(test source code)
└─ pages/	(page objects)
└─ fixtures/	(test fixtures)
└─ utils/	(helper utilities)
└─ testdata/	(test data files)
└─ api/	(API helpers)
└─ reports/	← saved to host
└─ allure-results/	← saved to host
└─ allure-report/	← saved to host
└─ custom-report/	← saved to host
└─ test-results/	← saved to host

Key idea: The container runs the tests. All reports and screenshots are automatically saved to your computer (bind-mounted volumes), so you can view them even after the container stops.

Files in This Project

File	What It Does
<code>Dockerfile</code>	Defines the Docker image — starts from <code>playwright:v1.62.0-noble</code> , installs npm dependencies, copies source code, sets up the entrypoint.
<code>docker-compose.yml</code>	Orchestrates the container — tells Docker which image to build, which files to mount, which environment variables to use, and which test suite to run.
<code>docker-entrypoint.sh</code>	A smart script that translates a simple suite name (like <code>sanity</code> or <code>api</code>) into the correct <code>npm run</code> command. You never need to edit this.
<code>.dockerignore</code>	Tells Docker which files to skip when building the image (e.g., <code>node_modules</code> , <code>.env</code> , old reports). Keeps the image small and secure.
<code>.env.example</code>	A template showing all available environment variables. Copy it to <code>.env</code> and fill in your values.

Prerequisites

Before you begin, make sure you have these installed:

1. **Docker Desktop** — [Download for Windows](#) | [Download for Mac](#)
 - After installing, launch Docker Desktop and wait for the whale icon to stop animating.
2. **Git** — [Download Git](#)
3. At least **4 GB** of free disk space (the Docker image is ~1.6 GB).

Verify Docker is working

Open a terminal (PowerShell on Windows, Terminal on Mac/Linux) and run:

```
docker --version
```

You should see something like: `Docker version 24.0.7, build afdd53b`

```
docker info
```

This should print system information without errors. If you get a permission error on Linux, see the [Troubleshooting](#) section.

Step-by-Step Setup

Step 1: Open a terminal in the project folder

Windows (PowerShell):

```
cd C:\Automation\POC_Playwright_opencart_AI_2
```

macOS / Linux:

```
cd /path/to/POC_Playwright_opencart_AI_2
```

Step 2: Create your environment file

The project uses a `.env` file to store settings like URLs and login credentials. A template already exists — you just need to copy it:

Windows (PowerShell):

```
Copy-Item .env.example .env
```

macOS / Linux:

```
cp .env.example .env
```

Now open `.env` in any text editor and fill in the values. For example:

```
# 🌐 WEB APP URL
WEB_APP_URL=https://tutorialsninja.com/demo/

# 🗝️ WEB APP LOGIN CREDENTIALS
APP_EMAIL=[REDACTED]@example.com
APP_PASSWORD=[REDACTED]

# 🌐 API BASE URL
API_BASE_URL=https://fakestoreapi.com
```

⚠️ Security note: The `.env` file is listed in `.dockerignore`, so it is **never baked into the Docker image**. It is only used at runtime when you run the container. This means your secrets stay safe.

Step 3: Build the Docker image

```
docker compose build
```

This command:

1. Reads `docker-compose.yml` and `Dockerfile`
2. Downloads the `mcr.microsoft.com/playwright:v1.62.0-noble` base image (first time only)
3. Installs all npm dependencies with `npm ci`
4. Copies the project source code
5. Tags the resulting image as `playwright-opencart`

⌚ **First build takes 2–5 minutes** depending on your internet speed. Subsequent builds are much faster because Docker caches layers.

Step 4: Verify the image was built

```
docker images playwright-opencart
```

You should see output like:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
playwright-opencart	latest	abc123def456	2 minutes ago	1.62 GB

Running Tests

All test commands follow the same pattern:

```
docker compose run --rm playwright-tests <suite-name>
```

Let's break this down:

- `docker compose run` — starts a one-off container (not the same as `docker compose up`)
- `--rm` — automatically deletes the container after tests finish (keeps your disk clean)
- `playwright-tests` — the service name defined in `docker-compose.yml`
- `<suite-name>` — which tests to run (see table below)

Available Test Suites

Command	What It Runs	Tag Filter
<code>docker compose run --rm playwright-tests</code>	All tests (no filter)	—
<code>docker compose run --rm playwright-tests all</code>	Same as above — all tests	—
<code>docker compose run --rm playwright-tests sanity</code>	Sanity tests only	@sanity
<code>docker compose run --rm playwright-tests regression</code>	Regression tests only	@regression
<code>docker compose run --rm playwright-tests web</code>	Web UI tests only	@web
<code>docker compose run --rm playwright-tests api</code>	API tests only	@api
<code>docker compose run --rm playwright-tests master</code>	Master suite	@master
<code>docker compose run --rm playwright-tests e2e</code>	End-to-end tests	@e2e
<code>docker compose run --rm playwright-tests datadriven</code>	Data-driven tests	@datadriven

1. Run All Tests

```
docker compose run --rm playwright-tests
```

Or explicitly:

```
docker compose run --rm playwright-tests all
```

This runs every test file in the `tests/` directory with no tag filter.

2. Run Sanity Tests

```
docker compose run --rm playwright-tests sanity
```

Sanity tests are quick checks tagged with `@sanity` — typically the most critical happy-path scenarios.

3. Run Regression Tests

```
docker compose run --rm playwright-tests regression
```

Regression tests (tagged `@regression`) are a broader set that verifies existing functionality hasn't broken.

4. Run Web Tests

```
docker compose run --rm playwright-tests web
```

Runs all browser-based UI tests tagged with `@web`.

5. Run API Tests

```
docker compose run --rm playwright-tests api
```

Runs all API-level tests tagged with `@api`. These are typically faster since they don't need a browser.

6. Run a Specific Test File

```
docker compose run --rm playwright-tests all -- tests/web/login.spec.ts
```

The `--` separates the suite name from Playwright's own arguments. Everything after `--` is passed directly to Playwright.

7. Pass Extra Playwright Options

You can pass any Playwright CLI flag:

```
# Run web tests in headed mode (see the browser window)
docker compose run --rm playwright-tests web -- --headed

# Run with fewer parallel workers
docker compose run --rm playwright-tests api -- --workers 2

# Run with more retries on failure
docker compose run --rm playwright-tests sanity -- --retries 3

# Run a specific project (e.g., chromium)
docker compose run --rm playwright-tests all -- --project=chromium
```

Passing Environment Variables

There are three ways to pass environment variables to the container. Choose whichever suits your workflow.

Using the `.env` file

This is the simplest approach. The `docker-compose.yml` already has:

```
env_file:
  - .env
```

So any variable you put in `.env` is automatically available inside the container. Just edit `.env` and run:

Accessing Reports and Test Artifacts

After tests finish, all output is automatically saved to your computer because `docker-compose.yml` mounts these directories as **volumes** (shared folders between the container and your host).

Where to find everything

Host Directory	What's Inside	How to Open
<code>./reports/</code>	Playwright HTML report (<code>index.html</code>)	<code>npx playwright show-report reports/</code>
<code>./allure-results/</code>	Raw Allure JSON data	Used by Allure CLI to generate a report
<code>./allure-report/</code>	Generated Allure HTML report	Open <code>index.html</code> in a browser
<code>./custom-report/</code>	Custom reporter output	Open <code>index.html</code> in a browser
<code>./test-results/</code>	Screenshots, videos, traces (on failure)	Browse the folder directly

View the Playwright HTML Report

```
npx playwright show-report reports/
```

This opens a nice interactive report in your browser where you can see:

- Which tests passed / failed
- Error messages and stack traces
- Screenshots of failures
- Video recordings (if any)
- Trace viewer for debugging

View the Allure Report

If you have Allure CLI installed on your host:

```
allure generate allure-results --clean -o allure-report
allure open allure-report
```

If you don't have Allure CLI, you can still browse the raw JSON files in `allure-results/`.

View Screenshots and Videos

Simply open the `test-results/` folder in your file explorer:

Windows:

```
explorer test-results
```

macOS:

```
open test-results
```

Linux:

```
xdg-open test-results
```

Complete Workflow Example

Here's a typical workflow from start to finish:

```
# 1. Navigate to the project
cd C:\Automation\POC_Playwright_opencart_AI_2

# 2. Create env file (first time only)
Copy-Item .env.example .env

# 3. Edit .env with your credentials (use any text editor)
notepad .env

# 4. Build the Docker image (first time or after dependency changes)
docker compose build

# 5. Run sanity tests to verify everything works
docker compose run --rm playwright-tests sanity

# 6. Run the full web suite
docker compose run --rm playwright-tests web

# 7. View the HTML report
npx playwright show-report reports/
```

CI/CD Integration

This Docker setup is designed to work seamlessly in CI/CD pipelines (Jenkins, GitHub Actions, GitLab CI, etc.).

Jenkins example

The project already includes a `Jenkinsfile.windows`. The Docker commands work the same way:

```
stage('Run Tests') {
    steps {
        bat 'docker compose build'
        bat 'docker compose run --rm playwright-tests regression'
    }
}
```

GitHub Actions example

```
- name: Run tests in Docker
  run: |
    docker compose build
    docker compose run --rm playwright-tests api
```

Troubleshooting

"docker: command not found"

Docker is not installed or not in your PATH. Install Docker Desktop and restart your terminal.

"Cannot connect to the Docker daemon"

Docker Desktop is not running. Launch Docker Desktop and wait for it to be ready.

On Linux: "permission denied" when running Docker

Add your user to the `docker` group:

```
sudo usermod -aG docker $USER
```

Then log out and back in (or restart your session).

Tests fail with "No tests found"

Make sure your test files have the correct tag. For example, to run `sanity` tests, your test must have:

```
test('should login successfully @sanity', async ({ page }) => {
  // ...
});
```

Or using the test annotation syntax:

```
test('should login successfully', {
  tag: '@sanity',
}, async ({ page }) => {
  // ...
});
```

"ENOENT: no such file or directory" for docker-entrypoint.sh

Make sure the file has Unix line endings (LF, not CRLF). On Windows, run:

```
# In the project root, using Git Bash or WSL:  
dos2unix docker-entrypoint.sh  
  
# Or in PowerShell with a simple fix:  
(Get-Content docker-entrypoint.sh) -join "`n" | Set-Content -NoNewline docker-entrypoint.sh
```

Port conflicts

If you get port binding errors, it means another service is using the port. Since we use `docker compose run` (not `up`), ports are not exposed by default, so this is unlikely.

Rebuild from scratch (clear all caches)

If you suspect a stale cache is causing issues:

```
docker compose build --no-cache
```

Quick Reference Card

```
# — Setup —————
Copy-Item .env.example .env          # Windows: create env file
cp .env.example .env                 # Mac/Linux: create env file
docker compose build                  # Build the image

# — Run Tests —————
docker compose run --rm playwright-tests      # All tests
docker compose run --rm playwright-tests sanity # Sanity
docker compose run --rm playwright-tests regression # Regression
docker compose run --rm playwright-tests web    # Web UI
docker compose run --rm playwright-tests api    # API
docker compose run --rm playwright-tests master # Master
docker compose run --rm playwright-tests e2e    # E2E
docker compose run --rm playwright-tests datadriven # Data-driven

# — With Extra Options —————
docker compose run --rm playwright-tests web -- --headed
docker compose run --rm playwright-tests api -- --workers 2

# — Environment Variables —————
docker compose run --rm -e WEB_APP_URL=https://example.com playwright-tests sanity

# — Reports —————
npx playwright show-report reports/
allure generate allure-results --clean -o allure-report
allure open allure-report

# — Cleanup —————
docker compose down          # Remove containers/networks
docker rmi playwright-opencart # Remove the image (frees ~1.6 GB)
```